Path Planning Using Constrained Delaunay Triangulation

Keshawn Boughton and Nadezhda Dominguez Salinas

December 16, 2025

Abstract

This project implements a computational geometry based path planning using a constrained Delaunay triangulation (CDT) for robot navigation. The classroom environment is obtained from an initial image. After detecting obstacles using computer vision sources and representing them as polygonal holes, we compute the CDT of the non obstructed space in the environment. Then, the centroids of the triangles from the CDT are used as nodes of a graph, and edges are constructed between centroids of adjacent triangles, to use as a potential navigation path. Using Dijkstra's algorithm on the centroid graph allows for a shortest path to be constructed from the start point to the target point, whilst remaining obstacle free. The results indicate that the centroid based shortest path is more efficient than navigating through all triangles contained in the CDT.

1 Introduction

In robotics, heavy emphasis is placed on choosing efficient path finding algorithms for environments containing obstacles. A robot must successfully navigate through a classroom environment by detecting and avoiding collision with objects such as desks, to move from it's start point to it's target destination. The main challenge in path planning is determining the strategies for the most efficient and reliable methods to use, especially when many obstacles are present in an environment. Classical path planning methods such as visibility graphs and the A* algorithm often lead to inefficient or bad time complexity in computation. Therefore, we used computational geometry as an alternative approach to attempt to reduce complexity.

In the project, we investigate using a constrained Delaunay triangulation (CDT), to divide a classroom environment into free space (the area excluding obstacles) and triangulate it. Then, once the CDT has been computed, each of it's triangle's centroid becomes a node in a graph. The adjacent triangles then create edges in this graph, and Dijkstra's algorithm is applied to compute the shortest path between two points within the free space. Our hypothesis is that the path created from our method will navigate through a reduced number of triangles in the CDT than the total triangulation, resulting in efficient path planning.

2 Background and Definitions

2.1 Polygonal Regions containing Polygonal Holes

A polygon is a closed region of a plane that contains a finite number of edges that form a closed curve without intersecting itself. The environment in which this project revolves is a polygon: a 4-edged polygon representing the classroom. Classrooms usually contain desks, tables, chairs and other miscellaneous items. The set of items will be notated as H and are represented as convex hulls in the rectangular classroom. To make H separate from the remaining space in the classroom, the items in H are represented as polygonal holes. This makes the environment a 4-edged region with polygonal holes. The question now becomes: how would the parts of the environment which can be traversed be indicated?

2.2 Triangulations

A diagonal of the environment is a line segment that (1) connects two vertices in the environment and (2) lies in the interior (except at its endpoints). In this case, the interior of the environment does not include the polygonal holes representing the items H. For the purposes of this project, the diagonals exist solely to partition the environment through which a robot can traverse. With the maximal quantity of non-crossing diagonals in the environment, the environment will be triangulated. In other words, these diagonals can split the environment into triangles. The location of diagonals indicate where in the environment the robot can traverse. But could every polygonal region with polygonal holes be triangulated?

It is known that all polygons are able to be triangulated. Suppose a different polygon P, which does not represent our environment, as P contains no polygonal holes. Given that P has k vertices, and $k \geq 3$, P can be triangulated. If another point were added to P, P would have $k + 1$ vertices. P can still be triangulated. Focus on three consecutive vertices $a, b, c \in P$. If a diagonal were made of $\overline{ac}$, P would be divided into a triangle and a polygon with k vertices. It is already known that this k-vertex polygon can be triangulated, meaning P can also be triangulated.

But is this the case for polygonal regions with polygonal holes? The main aspect of these regions to note is that they can be divided into smaller regions using diagonals, which are polygons. Therefore, the environment, regardless of the number or complexity of the polygonal holes, can also be triangulated.

2.3 Delaunay Triangulations and Constrained Delaunay Triangulation

A Delaunay triangulation is a triangulation for a set of points, which requires the empty circumcircle property and maximizes the minimum angle for any triangulation. For environments that contain obstacles, a regular Delaunay triangulation is not viable as the triangulation may cross over obstacle boundaries. Therefore a constrained Delaunay triangulation (CDT) is needed, to avoid obstacle boundary and triangulation intersections. The CDT takes into account the obstacle polygon's boundary edges when forming the triangulation, then computes the triangulation with these constraints added. Thus the CDT provides a triangulation for the classroom containing obstacles, while respecting boundaries and being as close to a Delaunay triangulation as possible.

3 Related Works

3.1 Path Planning Algorithms Comparison

Al-Zubaidi, Zaid M., et al (2023)[1] provided an overview of the commonly used path-planning algorithms applied on pick-and-place robots used in production lines. The authors compare the algorithms of A*, Rapidly Exploring Random Tree (RRT), and Probabilistic RoadMap (PRM) based on their computational complexity, path-length, and obstacle avoidance in the controlled environments in eight case studies. The results showed that there is not one optimal algorithm for path-planning, and instead, each has its trade off in terms of path-length and computational complexity. The authors found that RRT gives the longest path for all case studies, PRM had a larger computation time than others while consistently outputting short paths, and A* provides a reasonable computation time for path-planning yet it explores the most area in comparison to the other algorithms.

The article served as a good basis in assessing trade-offs between algorithms and which is best suited based on desired efficiency or area coverage of a robot used in path planning.

3.2 Using Voronoi Diagrams for Dynamic Environment Pathfinding

Ayawli, Ben Beklisi Kwame, et al (2019)[2] proposed a different path-finding algorithm, except this one also works for dynamic environments. Dynamic environments still prove to be an issue. To properly mark the environment for path planning, a Voronoi Diagram has been used. Using Voronoi Diagrams (VT), and Computational Geometry Technology (CGT), they have proposed an algorithm for dynamic motion planning. The main purpose of this paper was to make two contributions. The first was to mark moving objects based on their positions, distances, orientations, and velocities to readjust their planning. The second was to make an algorithm for computing path readjustments by marking parts of its environment. This was to determine which parts of the environment needed the most attention: the parts which contained moving objects. These moving objects were marked with different "collision threat levels".

The movements of its obstacles were measured in relation to the robot at all times. Its kinetics were measured using a lot of linear algebra and physics that consider the robot's turning angles and an obstacle's orientation in relation to the robot. Then, using CGT, morphological dilation (MD), VD, A* and Cubic Spline Interpolation (CSI), the robot's path was readjusted accordingly. The VD was used to mark the environment. MD was used to ensure a safe distance between the robot and its obstacles. CSI was used to smooth the robot's path plans and path adjustments. The algorithm started with the robot making an initial path, smoothed out using CSI. Then, given the moving obstacles and their distances to the robot, the path were readjusted accordingly.

This article is relevant to the project as this paper deals with a more advanced version of our problem. Demonstrating the use of computational geometry in application to navigation of robots and path planning.

3.3 Obstacle Avoidance Path Planning Based on Delaunay Triangulation

Jan, Gene Eu, et al (2021)[3] proposed a simple path-finding algorithm. This algorithm will be used in this project. Finding the shortest path between 2 points in a polygon with polygonal obstacles (or holes, in general cases) is a challenging yet important question to answer. Usually, there exists more than one path between 2 points. Which path is the shortest? This is where path planning algorithms come in. A* and Dijkstra's algorithms are the most common approaches when it comes to actually constructing a path. Dijkstra's is the easiest to implement and is more efficient than A* when points are not evenly distributed. But cell decomposition is also important. The most common type of cell decomposition is trapezoidal decomposition, which vertically divides the plane into smaller but polygonal spaces.

For environments where accessible spots aren't obvious or discretely explicit, it helps to add vertices to those accessible spots. This plot of vertices on the environment is known as a Roadmap. To make this Roadmap, there are 3 main approaches: Visibility Graph, Voronoi Diagram and Delaunay Triangulation. The Visibility Graph is the best at finding the shortest path given these vertices, but its computational complexity is $O(n^3)$, where n is the number of vertices. It is not very efficient for a large n. This paper proposed an algorithm with $O(nlog(n))$.

Given an environment with polygonal obstacles, the algorithm goes as follows: 1) Make a constrained Delaunay Triangulation of the image given these obstacles. 2) Calculate the centroids of the triangles. 3) Connect these centroids of all adjacent free spaces to construct a graph. 4) Apply Dijkstra's algorithm to this graph.

Their investigation serves as the basis for our methodology throughout our project, and provided a guide to building the centroid graph from the CDT, and then applying Dijkstra's as the path-planning algorithm.

4 Methods

4.1 Detecting Obstacles in the Classroom Environment

The first step in our process was not mentioned in the Related Works. But this step was crucial to constructing the environment, as it detected H in the form of convex hulls. The input was an image of the top-view of a classroom or classroom model. Given this image, the purpose of this step was to detect the items in H, as they were the items to avoid in the path-finding process. All attempts to execute this step used Python's OpenCV. The first method used "thresholding". This transformed the input to a binary image, where each pixel was black or white. The color of a given pixel depended on whether the pixel's color intensity surpassed a certain threshold. Although this detection method worked for some classroom objects, it was more sensitive to glare present in the image. Consider the image on the left of Figure 1. There is a glare present throughout the top of the image. This considerably affects object detection using this method. To the right are the polygonal holes detected using this method. Notice how none of the individual desks below the second row are detected.

To better detect elements in H, another method had to be used. This is where Canny Edge detection was introduced. Canny Edge detection detects parts of the image that have the greatest change in color intensity. It also makes a binary image, but its conditions are

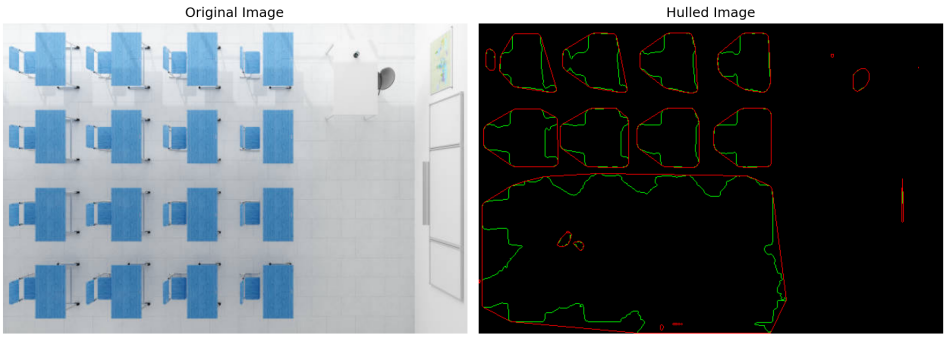


Figure 1: The left side presents the original image. The right side shows what obstacles were detected using thresholding.

more constrained. This requires two thresholds: lower threshold a and upper threshold b. If the intensity of a given pixel is higher than b, the pixel is classified as part of an edge. If it is less than a, the pixel is not part of an edge. If it is between a and b, it is part of an edge only if it is connected to a pixel with higher intensity than a. This detects objects in classroom environments better than regular thresholding, despite the glare present in many of them. Most of the objects present in the classroom were detected as edges in the binary image. While this method was not perfect, it did a better job in detecting H than the previous method. This is displayed in Figure 2.

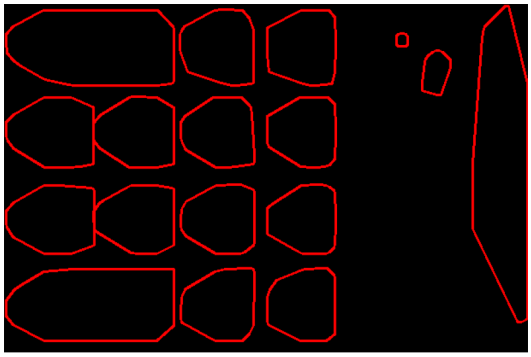


Figure 2: The same classroom, except objects were detected using Canny Edge Detection

Even though the items in H were more accurately detectable, they still had to be indicated by polygons. To make these polygons, the image was run through a different detector that listed the Convex Hulls around the detected edges. While most of the items in H were detected by these edges, their hulls were not the only hulls detected. If all of the detected hulls were included in the final environment, it would make the rest of the algorithm implementation more computationally expensive than necessary.

Python's OpenCV filters the hulls by hierarchy. This hierarchy indicates which of the hulls are inside other hulls. Fortunately, all of the necessary hulls were part of the same hierarchy. So, the object detection algorithm concluded by finding the one hierarchy that

contained all of the necessary objects and excluded the remaining hulls.

4.2 Create Constrained Delaunay Triangulation

The next step of the algorithm was to triangulate the environment, specifically using a Constrained Delaunay Triangulation (CDT). This step involved excluding the polygons that represented the items in H from the triangulation. It was executed using Python's Shapely. This library includes creating their own polygon objects and geometric sequences. It also includes an operation for implementing a CDT on the polygon object, which returns a geometric sequence of triangles. But how were the hulls representing the items in H excluded from the triangulation? When creating polygons using Shapely, an optional state of the polygon could include the holes in the environment. Here, the polygons representing H were added.

4.3 Computing CDT Centroids and Building Centroid Graph

To form the navigation graph, using Shapely's ".centroid" operation, the centroids were computed from the triangles of the CDT. These centroids served as the nodes of the navigation graph constructed using the Networkx package.

To build the centroid graph, if two triangles in the CDT were adjacent, then they shared a boundary edge. For each adjacent pair of triangles, we test that the line segment between centroids do not intersect any item in H, the obstacle convex hulls. If the line segment does not intersect the convex hull edges, then create a graph edge, whose weight is the Euclidean distance between the centroids. The result is a graph whose edges are within the environment and do not intersect the obstacle's edges.

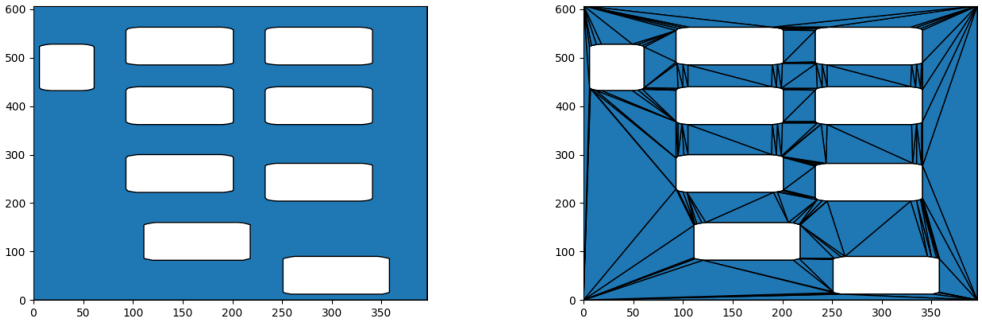
4.4 Path Finding Algorithm: Dijkstra's

Using this navigation graph, the robot's start and target positions are mapped to the centroids closest to these two points. Then, implementing Dijkstra's algorithm on the centroid graph provides the shortest path, and we store the centroids in order of the path. Finally, we plot the shortest path using the centroids to visualize the route from the start to the target point in the classroom. This results in an obstacle free shortest path to feed into a robot's navigation file.

5 Results

The project successfully rendered an efficient shortest path from an image of a classroom model with useful obstacle detection methods. In the following, output from our pipeline will be interpreted in the context of using Computational Geometry to retrieve our path.

The first step of our design resulted in the polygonal construction of the classroom from the image. The program was able to convert desks and chairs and represent their convex hulls which were then represented as polygonal holes inside the classroom rectangle boundary. The result is able to produce the classroom environment with the polygon constraints. In Figure 1 observe the outer classroom boundary and the various interior polygons representing the obstacles inside, demonstrating the free space to be used for triangulation.



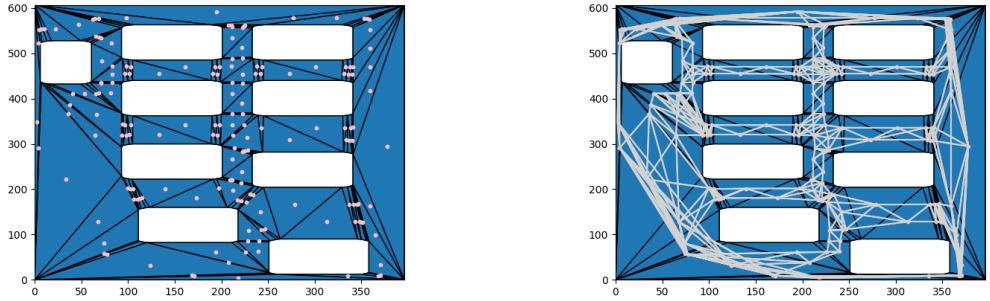
(a) The initial classroom model

(b) CDT of the classroom model

Figure 3: The classroom model and its constrained Delaunay triangulation visualized using Python's Shapely

Then, observe in Figure 3b the output of running Shapely's CDT algorithm onto our environment, as it produces a proper triangulation which does not cross obstacle boundaries. This provides evidence that the triangulation was successful in identifying the constraints. An interesting observation to make is that obstacles in the classroom with fewer distances between each other contained more triangles than that of the free space.

Furthermore, plotting the centroids (shown in pink) on the CDT provided a visualization of the nodes in the navigation graph. Then following our pipeline, the edges connected the centroids of adjacent triangles, as shown in Figure 4b, without intersecting obstacle edges. These edges capture the variability of paths that can be formed throughout our environment, where there is no collision with obstacles. This confirms the successful creation of an obstacle-free navigation graph, which can be used for more efficient path-planning.



(a) Centroids of the triangles of the CDT

(b) Centroid Navigation Graph

Figure 4: Classroom model showing centroid-based representations

The final implementation, shown in Figure 5, demonstrates the shortest path created by using Dijkstra's algorithm on the centroid navigation graph, where the shortest path is able to navigate around the obstacles. This path also used the smallest quantity of centroids in

comparison to the total number of centroids in the CDT. In Figure 4, there are 168 triangles in the CDT of the classroom, with the same number of centroids. The shortest path computed between the start and end centroids contained only 17 centroids. This confirms efficiency when path planning and successfully used computational geometry techniques as a different path finding strategy, which were suitable for our small scaled classroom environments. Further, the actual implementation of a physical robot navigating this shortest path was not possible, due to the Lego Robots being older models and not compatible to be programmed for a real environment. In future work, an implementation using a different robot model would be ideal, and require additional computation to account for angles between the centroids in the shortest path, and speed for the robot to navigate the free space. Our full implementation and code is documented and available on the following, GitHub(click here).

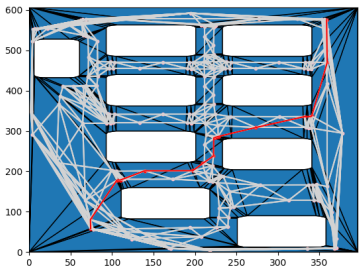


Figure 5: Shortest path computed between two points, based on centroid navigation graph

6 Conclusion

The project implemented a CDT based path planning strategy that effectively produced a shortest path for a potential robot to use when given a classroom environment with obstacles. This planning strategy was solely programmed in Python, using its various libraries. Our environment involved the traversable space in a classroom, which excluded the set H of obstacles in the classroom. This environment was constructed using Python's OpenCV. Then the environment was triangulated and used to make a viable path using Python's Shapely. Through identifying obstacles in the environment, computing the constrained triangulation, building a centroid graph, and applying Dijkstra's algorithm, we obtained an optimum path for a robot to navigate a classroom.

References

[1] Zaid M. Al-Zubaidi, Ali A. Abdulrahman, and Atheer Al-Dulaimi. Comparison of path planning algorithms for pick-and-place robots in industrial environments. *Robotics and Autonomous Systems*, 162:104327, 2023.

[2] Ben Beklisi Kwame Ayawli, Kofi Appiah, and Stephen Opoku. A voronoi diagram-based path planning algorithm for autonomous mobile robots in dynamic environments. *Applied Sciences*, 9(18):3811, 2019.

[3] Gene Eu Jan, Chien-Hung Lin, and Chih-Yung Hsu. Obstacle avoidance path planning based on delaunay triangulation for mobile robots. *IEEE Access*, 9:137894–137905, 2021.